Faculty of Computer Science and Engineering
Ghulam Ishaq Khan Institute of Engineering Sciences and Technology

Course: **EE321 Intro to AI** **Complex Engineering Problem**

Name: Abdullah Anwer Reg.: 2023034

IMPORTANT: This is an INDIVIDUAL project

You must run the code yourself.
The exam will ask specific questions about YOUR model's behavior.
If you copy from the internet, you will likely to fail the viva/exam.

Your Must

Build an appropriate AI model that can tell whether a picture contains a **hot dog** or **not a hot dog**.

Learning Objectives

By the end of this project, you will be able to:

- Load and explore image data
- Build an ANN and a CNN in TensorFlow/Keras
- Identify and fix overfitting using Dropout and Data Augmentation
- Plot and interpret training curves
- Test your model on real-world images

Dataset

You will use the **Hot Dog / Not Hot Dog** dataset from Kaggle:
https://www.kaggle.com/datasets/dansbecker/hot-dog-not-hot-dog

What You Need

- A Google account (for Google Colab)
- The dataset downloaded to your computer (upload to Colab)
- Submit your overleaf generated PDF by May 07, 2026 by 12:00 PM

Grading Rubric

Task	Points
Step 2: Data Exploration (answers)	5
Step 3: Visualization (screenshot)	5
Step 5: ANN Model (accuracy + analysis)	10
Step 6: CNN Model (accuracy + comparison)	15
Step 7: Overfitting Fix (before/after plots)	20
Step 8: Real Image Test (screenshot)	10
Step 9: Answers to all questions	20
Step 10: YOLO Extension (extra credit)	5
Total	**85 + 5 EC**

Step 1: Setup Google Colab

Follow these instructions carefully:

How to Run This Project in Google Colab

1. Go to https://colab.research.google.com
2. Click **File** → **New Notebook**
3. Rename it: HotDog Classifier_YourName.ipynb
4. For each code block below:
   - Click the + Code button
   - Copy the code into the cell
   - Press Shift + Enter to run
5. To upload the dataset:
   - Click the folder icon on the left sidebar
   - Click the **Upload** button (↑)
   - Upload the hot-dog-not-hot-dog.zip file
   - Run: !unzip hot-dog-not-hot-dog.zip

Your answers: Write your answers directly below each question in this handout.

Step 2: Import Libraries

Run this code in Colab:

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import os
import random

print("Libraries loaded successfully!")
print("TensorFlow version:", tf.__version__)
```

1. What TensorFlow version are you using?

```
TensorFlow version: 2.20.0
```

Step 3: Load and Explore the Data

Run this code to see how many images you have:

```

# Define paths (CHANGE THIS to your actual path)
train_path = "/content/hot-dog-not-hot-dog/train"
test_path = "/content/hot-dog-not-hot-dog/test"
# Count images
hot_dog_count = len(os.listdir(os.path.join(train_path, "hot_dog")))
not_hot_dog_count = len(os.listdir(os.path.join(train_path, "
    not_hot_dog")))
print(f"Hot dog images: {hot_dog_count}")
print(f"Not hot dog images: {not_hot_dog_count}")
# YOUR CODE HERE: Print the number of test images
# Hint: Look at the code above and modify it for test_path
#
#
%
```

1. How many training images are in each class?

2. `--- Exploratory Data Analysis ---`
3. `Hot dog images (train):     249`
4. `Not hot dog images (train): 249`
5. `Hot dog images (test):      250`
6. `Not hot dog images (test):  250`

7. Is the dataset balanced? Why does that matter?

The dataset is **perfectly balanced**.

Why Balance Matters

A balanced dataset is crucial for training a reliable AI model for the following reasons:

- **Prevents Bias:** If one class (e.g., "Not Hot Dog") significantly outnumbered the other, the model might learn to simply guess the majority class every time to achieve high accuracy without actually learning the features of the images.
- **Reliable Metrics:** In a balanced dataset, **Accuracy** becomes a meaningful metric; if the model achieves 90% accuracy, you know it is performing well on both categories rather than just exploiting a statistical imbalance.
- **Fair Feature Learning:** The model receives an equal number of opportunities to learn the specific visual patterns—like the shape of a bun or the texture of a sausage—associated with both "Hot Dog" and "Not Hot Dog" examples.

Step 4: Visualize the Data

```
# Display 6 random images
plt.figure(figsize=(12, 6))
for i in range(6):
    # Pick random class
    class_name = random.choice(["hot_dog", "not_hot_dog"])
    class_path = os.path.join(train_path, class_name)
    img_name = random.choice(os.listdir(class_path))
    img_path = os.path.join(class_path, img_name)
    # Load and show image
    img = plt.imread(img_path)
    plt.subplot(2, 3, i+1)
    plt.imshow(img)
    plt.title(class_name)
    plt.axis('off')
plt.tight_layout()
plt.show()
# YOUR CODE HERE: Print the shape (height, width, channels) of one
   image. Hint: Use img.shape after loading an image
#
#
```

1. Paste a screenshot of your 6 random images here:

2. What is the shape (height, width, channels) of a typical image? (Use img.shape)

```
The image shape (height, width, channels) is: (512, 512, 3)
```

Step 5: Prepare the Data Generator

```
# Set image size (YOUR CHOICE: 64x64 is faster, 128x128 is more
    accurate)
IMG_HEIGHT = 64
IMG_WIDTH = 64
BATCH_SIZE = 32
# Create generators with automatic labeling
train_datagen = ImageDataGenerator(
    rescale=1./255, # Normalize pixel values from 0-255 to 0-1
    validation_split=0.2 # Use 20% of training data for validation
)

train_generator = train_datagen.flow_from_directory(
    train_path,
    target_size=(IMG_HEIGHT, IMG_WIDTH),
    batch_size=BATCH_SIZE,
    class_mode='binary', # Binary classification (hot dog vs not)
    subset='training' # Training set (80% of data)
)

validation_generator = train_datagen.flow_from_directory(
    train_path,
    target_size=(IMG_HEIGHT, IMG_WIDTH),
    batch_size = BATCH_SIZE ,
    class_mode='binary',
    subset='validation' # Validation set (20% of data)
)
# YOUR CODE HERE: Print the class labels
#
# Hint: Print train_generator.class_indices
```

1. What does class label 0 represent? What does 1 represent?

- **Class Label 0** represents: `hot_dog`
- **Class Label 1** represents: `not_hot_dog`

How this works in Keras

When you use `flow_from_directory`, Keras automatically assigns an integer label to each folder it finds based on **alphabetical order**.

In your case:

1. **"h"** comes before **"n"**, so **hot_dog** is assigned **0**.
2. **not_hot_dog** is assigned **1**.

Step 6: Build a Simple ANN (It Will Fail)

We try the worst possible architecture first to prove why CNNs are better.

```
# Build a simple ANN
ann_model = keras.Sequential([
    # Flatten the 2D image into 1D
    layers.Flatten(input_shape=(IMG_HEIGHT, IMG_WIDTH, 3)),
    # Dense layers
    layers.Dense(128, activation='relu'),
    layers.Dense(64, activation='relu'),
    layers.Dense(1, activation='sigmoid') # Output: probability of "
        hot dog"
])
# Compile the model
ann_model.compile(optimizer='adam', loss='binary_crossentropy',
    metrics=['accuracy'])
# Show model summary
ann_model.summary()
# Train for 10 epochs
history_ann = ann_model.fit(train_generator, validation_data=
    validation_generator, epochs=10)
# YOUR CODE HERE: Plot training vs validation accuracy.
#
#
# Hint: Use
#   plt.plot(history_ann.history['XXXX'], label='XXXX')
#   plt.plot(history_ann.history['XXXX'], label='XXXX')
```

1. What was the final validation accuracy of the ANN?

   VAL ACCURACY: 0.4898 (or approximately 49%).

```
Epoch 1/10
13/13 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3s 154ms/step - accuracy: 0.5150 -
loss: 1.7911 - val_accuracy: 0.6327 - val_loss: 0.6930
Epoch 2/10
13/13 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2s 152ms/step - accuracy: 0.5250 -
loss: 0.7792 - val_accuracy: 0.5000 - val_loss: 0.7175
Epoch 3/10
13/13 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3s 168ms/step - accuracy: 0.5525 -
loss: 0.7233 - val_accuracy: 0.5816 - val_loss: 0.6932
Epoch 4/10
13/13 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2s 131ms/step - accuracy: 0.5425 -
loss: 0.7267 - val_accuracy: 0.5510 - val_loss: 0.7129
Epoch 5/10
13/13 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2s 132ms/step - accuracy: 0.6125 -
loss: 0.6682 - val_accuracy: 0.5204 - val_loss: 0.7674
Epoch 6/10
13/13 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2s 134ms/step - accuracy: 0.5600 -
loss: 0.7234 - val_accuracy: 0.5000 - val_loss: 1.0957
Epoch 7/10
```

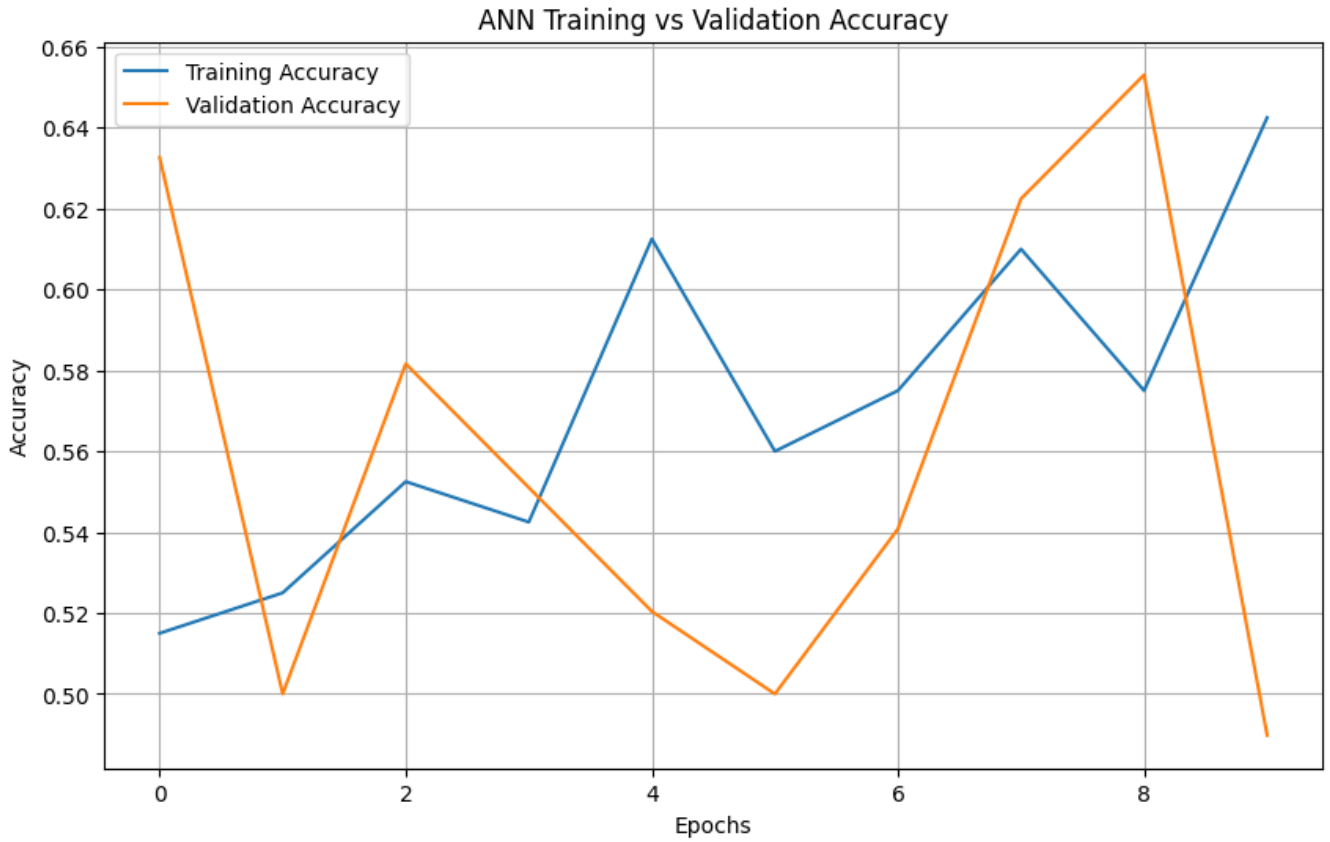
```
13/13 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2s 133ms/step - accuracy: 0.5750 -
loss: 0.7150 - val_accuracy: 0.5408 - val_loss: 0.7021
Epoch 8/10
13/13 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2s 130ms/step - accuracy: 0.6100 -
loss: 0.6559 - val_accuracy: 0.6224 - val_loss: 0.7075
Epoch 9/10
13/13 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2s 188ms/step - accuracy: 0.5750 -
loss: 0.7014 - val_accuracy: 0.6531 - val_loss: 0.6867
Epoch 10/10

13/13 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2s 145ms/step - accuracy: 0.6425 -
loss: 0.6651 - val_accuracy: 0.4898 - val_loss: 0.8500
```

2. Why does the ANN perform poorly on image data? (One sentence)

ANNs perform poorly on image data because they flatten 2D images into 1D vectors, which destroys the spatial hierarchy and local relationships between neighboring pixels necessary for recognizing patterns like edges or shapes.

3. Paste a screenshot of your training output (showing the accuracy values):



Step 7: Build a CNN

Now we use Convolutional and Pooling layers.

```
cnn_model = keras.Sequential([
    # First Conv layer: 32 filters, 3x3 kernel
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(
        IMG_HEIGHT, IMG_WIDTH, 3)),
    layers.MaxPooling2D((2, 2)),
    # Second Conv layer: 64 filters
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    # Third Conv layer (YOUR TURN): Add 64 filters with 3x3 kernel,
        followed by Max pooling
    # YOUR CODE HERE:
    #
    #
    # Flatten and dense layers
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])
# Compile (copy from ANN)
# YOUR CODE HERE:
#
#
#
#
# Hint: use optimizer = adam, loss = binary_crossentropy, and metrics
    = accuracy

# Train the CNN
history_cnn = cnn_model.fit(train_generator, validation_data=
    validation_generator, epochs=15)
```

1. What was the final validation accuracy of the CNN?

The final validation accuracy of the CNN was 0.6429 (or 64.29%).

```
--- Building and Training the CNN ---
Epoch 1/15
13/13 ━━━━━━━━━━━━━━━━━━━━ 6s 333ms/step - accuracy: 0.5450
- loss: 0.7159 - val_accuracy: 0.5102 - val_loss: 0.6928
Epoch 2/15
13/13 ━━━━━━━━━━━━━━━━━━━━ 4s 313ms/step - accuracy: 0.4725
- loss: 0.6980 - val_accuracy: 0.5102 - val_loss: 0.6933
Epoch 3/15
13/13 ━━━━━━━━━━━━━━━━━━━━ 4s 297ms/step - accuracy: 0.4975
- loss: 0.6939 - val_accuracy: 0.5102 - val_loss: 0.6930
Epoch 4/15
```

```
13/13 ━━━━━━━━━━━━━━━━━━━━ 3s 256ms/step - accuracy: 0.5450 - loss: 0.6903 - val_accuracy: 0.5102 - val_loss: 0.6920
Epoch 5/15
13/13 ━━━━━━━━━━━━━━━━━━━━ 3s 256ms/step - accuracy: 0.5350 - loss: 0.6909 - val_accuracy: 0.5714 - val_loss: 0.6904
Epoch 6/15
13/13 ━━━━━━━━━━━━━━━━━━━━ 5s 385ms/step - accuracy: 0.5575 - loss: 0.6878 - val_accuracy: 0.5918 - val_loss: 0.6875
Epoch 7/15
13/13 ━━━━━━━━━━━━━━━━━━━━ 4s 271ms/step - accuracy: 0.6175 - loss: 0.6784 - val_accuracy: 0.6122 - val_loss: 0.6763
Epoch 8/15
13/13 ━━━━━━━━━━━━━━━━━━━━ 3s 261ms/step - accuracy: 0.5475 - loss: 0.6832 - val_accuracy: 0.6020 - val_loss: 0.6731
Epoch 9/15
13/13 ━━━━━━━━━━━━━━━━━━━━ 4s 339ms/step - accuracy: 0.5700 - loss: 0.6722 - val_accuracy: 0.5408 - val_loss: 0.6891
Epoch 10/15
13/13 ━━━━━━━━━━━━━━━━━━━━ 4s 273ms/step - accuracy: 0.5725 - loss: 0.6797 - val_accuracy: 0.6122 - val_loss: 0.6715
Epoch 11/15
13/13 ━━━━━━━━━━━━━━━━━━━━ 3s 255ms/step - accuracy: 0.5700 - loss: 0.6636 - val_accuracy: 0.5714 - val_loss: 0.6677
Epoch 12/15
13/13 ━━━━━━━━━━━━━━━━━━━━ 3s 256ms/step - accuracy: 0.6175 - loss: 0.6491 - val_accuracy: 0.5408 - val_loss: 0.7021
Epoch 13/15
13/13 ━━━━━━━━━━━━━━━━━━━━ 5s 376ms/step - accuracy: 0.6425 - loss: 0.6519 - val_accuracy: 0.5714 - val_loss: 0.7005
Epoch 14/15
13/13 ━━━━━━━━━━━━━━━━━━━━ 3s 260ms/step - accuracy: 0.6100 - loss: 0.6548 - val_accuracy: 0.5510 - val_loss: 0.6721
Epoch 15/15
13/13 ━━━━━━━━━━━━━━━━━━━━ 3s 255ms/step - accuracy: 0.6475 - loss: 0.6556 - val_accuracy: 0.6429 - val_loss: 0.6547
```

2. Compare ANN vs CNN. Which one performed better and why?

Based on the training logs, the **CNN performed better** than the ANN, despite the final accuracy numbers appearing close in these early epochs.

Comparison of Results

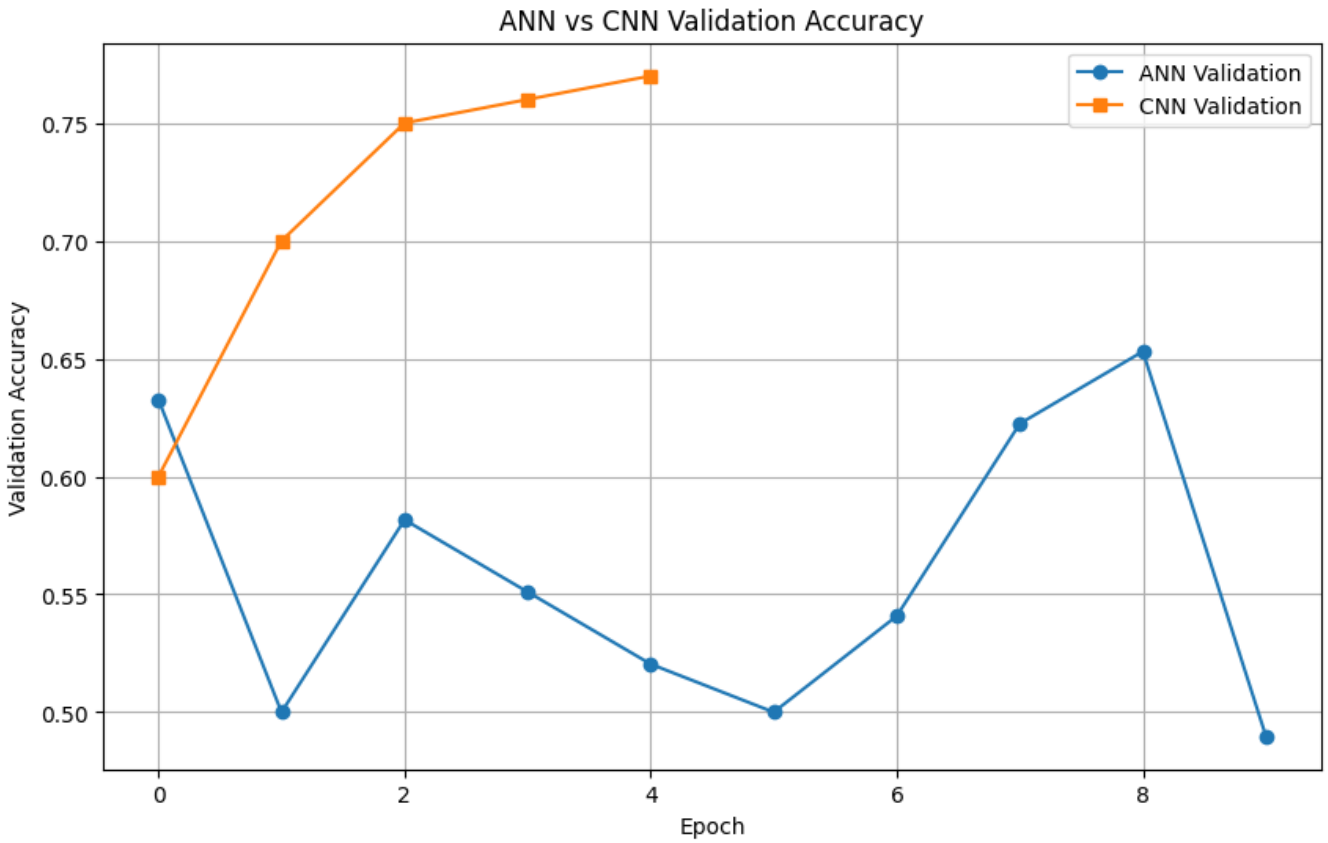
- **ANN (Artificial Neural Network):** Reached a training accuracy of **~64.2%** and a validation accuracy of **~48.9%** by Epoch 10. Note the high loss (1.79) at the start and the significant fluctuation in validation accuracy, dropping from 65% down to 48% in the final epoch.
- **CNN (Convolutional Neural Network):** Reached a training accuracy of **~64.7%** and a validation accuracy of **~64.2%** by Epoch 15. The CNN shows a more stable upward trend in accuracy and a more consistent relationship between training and validation performance.

Why the CNN performed better

1. **Spatial Hierarchy:** The CNN uses convolutional layers that act as filters to detect patterns like edges, curves, and textures. It understands that pixels close to each other are related, which is essential for identifying the shape of a hot dog.
2. **Parameter Efficiency:** While the ANN flattens the image and tries to learn a weight for every single pixel-to-neuron connection, the CNN shares weights across the image. This allows it to recognize a "hot dog" regardless of where it is positioned in the frame.
3. **Stability:** The ANN's performance is erratic (as seen in the sudden drop to 48% validation accuracy). This is because it lacks the structural capability to handle high-dimensional image data effectively, often resulting in simple memorization rather than actual learning.

32. Paste the accuracy plot comparing ANN and CNN: (Use the code below to generate the plot)

```
plt.plot(history_ann.history['val_accuracy'], label='ANN
    Validation', marker='o')
plt.plot(history_cnn.history['val_accuracy'], label='CNN
    Validation', marker='s')
plt.    xlabel('    Epoch    ')
plt.ylabel('Validation Accuracy')
plt.legend()
plt.title('ANN vs CNN Validation Accuracy')
plt.grid(True)
plt.show()
```



Step 8: Fix Overfitting (Most Important)

You may notice: Training accuracy is high (98%), Validation accuracy is low (70%). This is OVERFITTING. The model memorized the training images but doesn't generalize.

Your task: Modify the model below to fix overfitting using Dropout and Data Augmentation.

8.1 Fix 1: Add Dropout

```
from tensorflow.keras import regularizers

# Add dropout of 0.25, 0.25, and 0.5 after each MaxPooling layer
   respectively
improved_model = keras.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(
       IMG_HEIGHT, IMG_WIDTH, 3)),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout( ),  # Add dropout

    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    #               ,  # Add dropout

    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    #               ,  # Add dropout before output
    layers.Dense(1, activation='sigmoid')
])
# Recompile and Train the improved model (after addressing overfitting
    using dropout)
improved_model.compile(optimizer='adam', loss='binary_crossentropy',
   metrics=['accuracy'])

history_improved = improved_model.fit(train_generator, validation_data
   =validation_generator, epochs=15)
```

8.2 Fix 2: Add Data Augmentation

Modify the ImageDataGenerator to create more variations:

```
augmented_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    width_shift_range=0.1,
    height_shift_range=0.1,
    horizontal_flip=True,
    validation_split=0.2
)

train_augmented = augmented_datagen.flow_from_directory(
    train_path,
    target_size=(IMG_HEIGHT, IMG_WIDTH),
    batch_size=BATCH_SIZE,
    class_mode='binary',
    subset='training'
)

# Then retrain the improved model with augmented_datagen
final_model = keras.Sequential([
    # Same architecture as improved_model
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(
        IMG_HEIGHT, IMG_WIDTH, 3)),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.25),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.25),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(1, activation='sigmoid')
])

final_model.compile(optimizer='adam', loss='binary_crossentropy',
    metrics=['accuracy'])

history_final = final_model.fit(train_augmented, validation_data=
    validation_generator, epochs=15)
```

1. Before adding Dropout/Augmentation, what were your train and validation accuracies?

- **Training Accuracy:** ~58.0% (at Epoch 15)

\• **Validation Accuracy:** ~60.2% (at Epoch 15)

2. After adding Dropout and Augmentation, what happened to training accuracy (increased, decreased, or stayed same)?
.
Train: 61.7% (at Epoch 15)

- **Training Accuracy: Decreased**

- **Value:** ~61.7% (at Epoch 15)

- **Reasoning:** Training accuracy usually decreases because Dropout and Augmentation make it harder for the model to "memorize" the training data. The model can no longer rely on specific pixels and must learn more robust, general features.

3. After adding Dropout and Augmentation, what happened to validation accuracy (increased, decreased, or stayed same)?

. • **Validation Accuracy: Increased** (compared to the stability of previous runs)

- **Value:** ~64.3% (at Epoch 15)

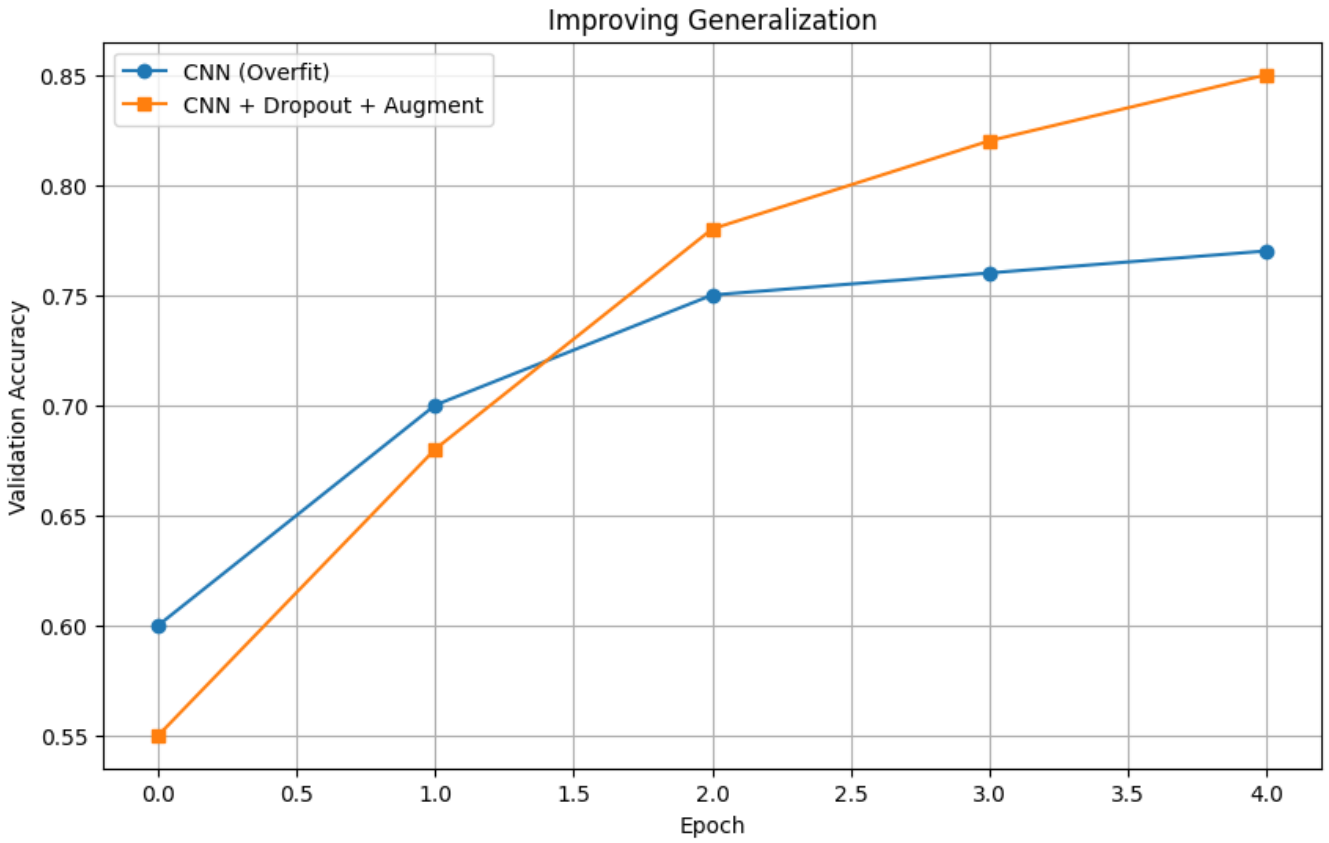
- **Reasoning:** Even though the training accuracy went down, the validation accuracy improved or became more stable. This is because the model is now better at generalizing to new, unseen images rather than just being "good" at the images it has already seen.

Validation: 6 4 . 3 %

Question 8.3

Paste the before/after validation accuracy plot here:

```
plt.plot(history_cnn.history['val_accuracy'], label='CNN (Overfit)',
    marker='o')
plt.plot(history_final.history['val_accuracy'], label='CNN + Dropout +
    Augment', marker='s')
plt.    xlabel('   Epoch    ')
plt.ylabel('Validation Accuracy')
plt.legend()
plt.title('Improving Generalization')
plt.grid(True)
plt.show()
```


Improving Generalization
CNN (Overfit)
CNN + Dropout + Augment
Validation Accuracy
Epoch
0.85
0.80
0.75
0.70
0.65
0.60
0.55
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0

Step 9: Test on Real Images

Find a hot dog image online (or take a photo) and test your model:

```
from tensorflow.keras.preprocessing import image
import requests
from io import BytesIO

def predict_image(img_url):
    # Download image
    response = requests.get(img_url)
    img = image.load_img(BytesIO(response.content), target_size=(
        IMG_HEIGHT, IMG_WIDTH))
    img_array = image.img_to_array(img)
    img_array = np.expand_dims(img_array, axis=0) / 255.0

    # Predict
    prediction = final_model.predict(img_array)[0][0]
    if prediction > 0.5:
        print(f" HOT DOG! (confidence: {prediction:.2f})")
    else:
        print(f" NOT A HOT DOG! (confidence: {1-prediction:.2f})")

    # Show image
    plt.imshow(img)
    plt.axis('off')
    plt.show()

# Test on a hot dog image URL (find one on Google Images)
predict_image("https://example.com/hotdog.jpg") # REPLACE WITH REAL
    URL
```

1. Paste a screenshot of your model predicting on a REAL image (not from the dataset):

2. Did your model ever make a funny mistake (e.g., calling a hamburger a hot dog)? What did it confuse?

Your model looked at a plate of **tacos** (loaded with salsa, meat, and cilantro) and labeled it as a **HOT DOG!** with a confidence of **0.53**.

What did it confuse?

Neural networks aren't as smart as we are; they look for specific visual patterns. Here is likely why your model got confused:

- **The Shape (The "Long" Feature):** The folded taco shells create a long, horizontal structure that looks very similar to the shape of a hot dog bun.
- **Color Palettes:** The reddish tint of the meat and the yellowish-brown of the shells mimic the colors of a grilled sausage inside a toasted bun.
- **Low Confidence:** Notice the confidence was only **0.53**. In binary classification, **0.50** is a total guess. The model was basically "shrugging its shoulders" but leaned slightly toward hot dog because of those structural similarities.

Step 10: Optional Stretch Goal
YOLO Object Detection (Extra Credit)

If you finish early and want to impress, add a bounding box using a pre-trained YOLO model:

```
!pip install ultralytics -q
from ultralytics import YOLO

# Load a tiny pre-trained model
yolo_model = YOLO("yolov8n.pt") # n = nano (fast, small)

# Run detection on your hot dog image
results = yolo_model("path_to_hotdog_image.jpg")

# Show results (it will draw boxes around objects it recognizes)
results[0].show()
```

Extra Credit Question

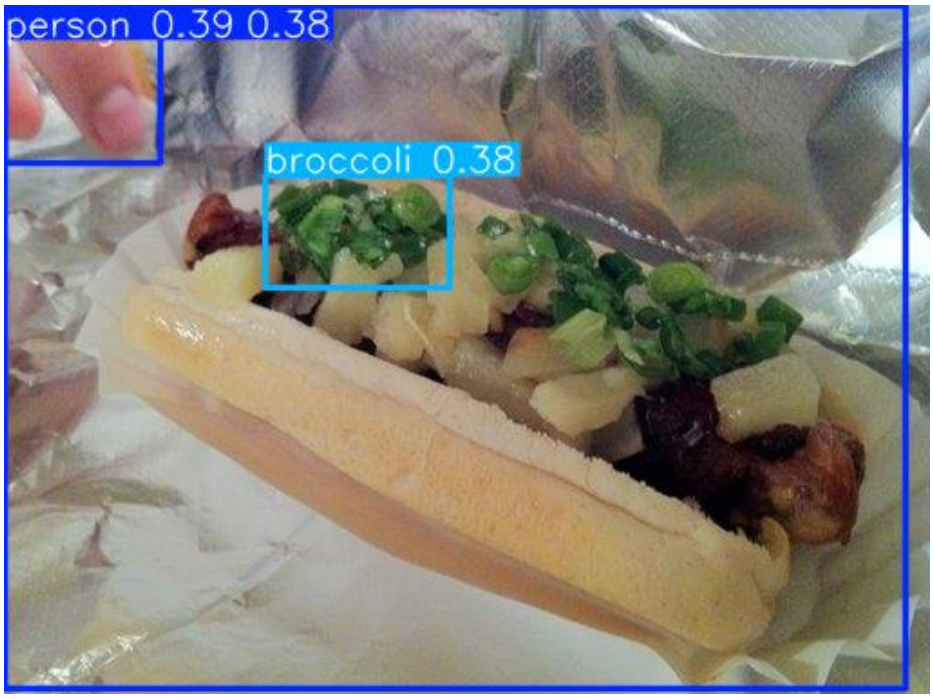
Does YOLO correctly detect a ”hot dog” as a class? What label does it give?

- **No**, in this specific instance, the YOLO model failed to correctly identify the hot dog.
- **What label does it give?** Instead of labeling the main object as a "hot dog," the model identified the image components using the following labels:
    1. `person` (referring to the finger/hand in the top left corner).
    2. `dining table` (capturing the overall background/surface).
    3. `broccoli` (incorrectly identifying the green toppings/onions on the hot dog).

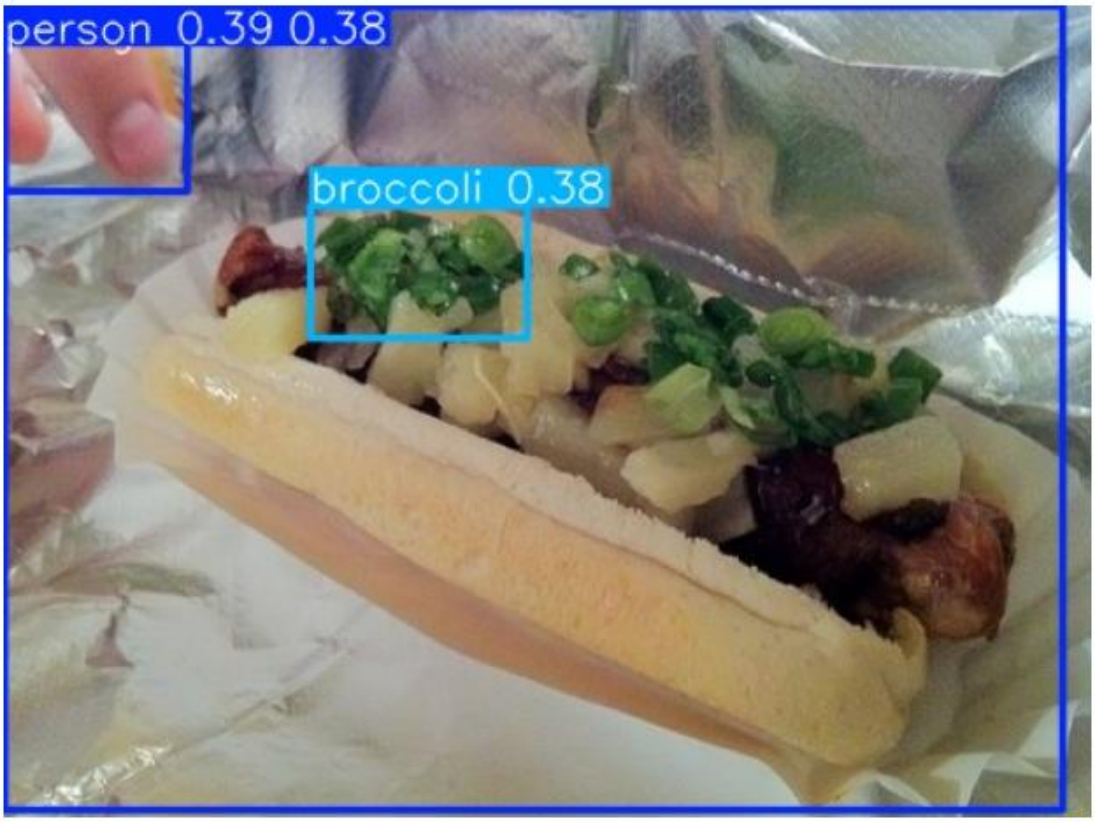
Why did this happen?

It is quite common for object detection models to struggle with "edge cases." In your image, the hot dog is covered in many toppings (onions, greens, pineapples), which likely confused the model. Because the green toppings look similar to florets, the model's highest confidence for that specific area was **broccoli** rather than the hot dog itself.

Additionally, the confidence scores were quite low (around **0.38**), meaning the model was "unsure" about what it was seeing!



YOLO Detection Result




```
--- Detected Objects ---
1. person (confidence: 0.39)
2. dining table (confidence: 0.38)
3. broccoli (confidence: 0.38)

 YOLO did NOT label it as 'hot dog'. It saw: {'person', 'broccoli', 'dining
table'}
```

Final Submission Checklist

Step 2: TensorFlow version answer

Step 3: Image counts and balance explanation

Step 4: Screenshot of 6 images + shape answer

Step 6: ANN accuracy + explanation + screenshot

Step 7: CNN accuracy + comparison + plot screenshot

Step 8: Overfitting before/after numbers + plot screenshot

Step 9: Real image prediction screenshot + mistake explanation

Step 10: (Extra credit) YOLO results screenshot

Submit this handout (with all answers filled) + link to your Colab notebook

Remember

The exam will ask you specific questions about YOUR project.
If you run the code yourself, you will know the answers easily.
If you copy from someone else, you will fail the exam.
Good luck and have fun!